



TEXAS A&M UNIVERSITY
ROBOTICS TEAM & LEADERSHIP EXPERIENCE

Week 3: Transformations in Python

TURTLE Incubator: SO-ARM Software Stack

Python and pip



Every useful library out there will not be prepackaged with your installation of Python

To install more, we use Python's package manager, **pip**

To install the libraries we need, open your terminal and type:

```
pip install numpy pandas matplotlib scikit-learn
```

Common Python Libraries



NumPy: Vector/Matrix manipulation

Matplotlib: Building Plots

Pandas: Tables and Spreadsheets

Sci-kit: Statistics and Machine Learning



- NumPy is the go-to library for anything related to vectors
- All the other libraries listed use NumPy under the hood
- Centers around its **ndarray** data structure
- These **ndarrays** have many useful features, such as
 - Transposing/reshaping matrices
 - Slicing the array



What is an Array?



- An array is a data structure for storing and retrieving a list of data
- NumPy's **ndarrays** can have as many dimensions as needed
- **ndarray** is different from Python's **list** in two major ways:
 - Homogeneity: All elements in an array must be of the same data type
 - Fixed size: Once created, the amount of elements in the array cannot change

These restrictions let **ndarrays** be much faster and memory efficient than traditional **lists**

1	5	2	0
---	---	---	---

A one-dimensional array of integers

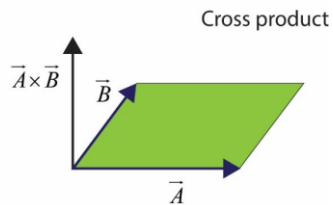
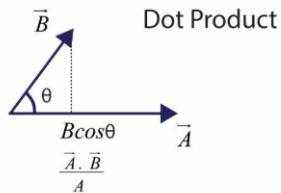
1	5	2	0
8	3	6	1
1	7	2	9

A two-dimensional array of integers

NumPy functionality



You can do many useful linear algebra operations with NumPy arrays like:



Dot and cross product

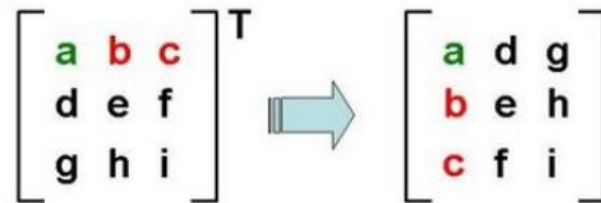
2	3
4	5

`matrix.flatten()`



2	3	4	5
---	---	---	---

Flattening matrices



Transposition/
Reshaping

Code Along: NumPy



1

Make your file

In VS Code, create a new file in your project folder.

2

Type along

Follow the next two slides line by line. Don't copy and paste: typing it is the point.

3

Check your output

Run it. Every print should match the comment next to it.

YOUR FILE

week3_numpy.py

RUN IT

```
python week3_numpy.py
```

Code Along: NumPy Arrays



```
import numpy as np

# SO-ARM joint angles (deg): pan, lift, elbow, wrist_flex, wrist_roll, gripper
joints = np.array([0, -45, 90, 30, 0, 10], dtype=float)
print(joints.shape, joints.dtype)           # (6,) float64

arm = joints[:3]                            # [ 0. -45.  90.]
gripper = joints[-1]                        # 10.0
joints_rad = np.deg2rad(joints)            # whole array at once, no loop

M = np.arange(1, 13).reshape(3, 4)
print(M[:, 2])                             # third column: [ 3  7 11]
print(M.T.shape, M.flatten())              # (4, 3) and 1D array
```

Arrays: one type, fixed size, sliced just like lists.

Code Along: NumPy Math



```
import numpy as np

a, b = np.array([1, 2, 3]), np.array([4, 5, 6])
print(np.dot(a, b))           # 32
print(np.cross(a, b))        # [-3  6 -3]

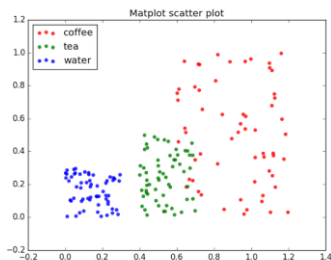
A = np.array([[1, 2], [3, 4]])
B = np.array([[0, 1], [1, 0]])
print(A * B)                 # element by element
print(A @ B)                 # real matrix multiply
```

Predict both * and @ answers before you run it! Check every output against the comments.

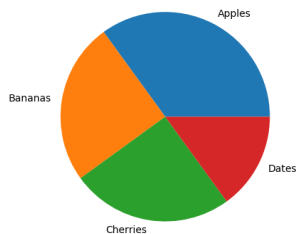


- Matplotlib is a very useful library that allows you to generate many different types of plots.
- Can generate plots from standard lists, NumPy arrays, Pandas tables

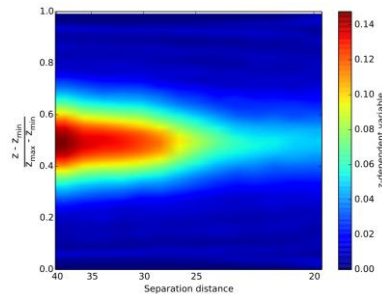
Available plot types



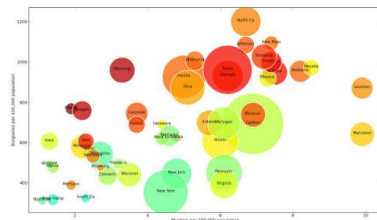
Scatterplots



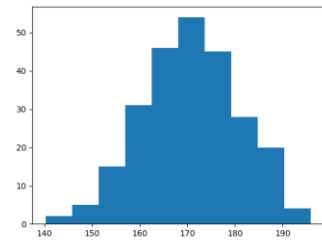
Pie charts



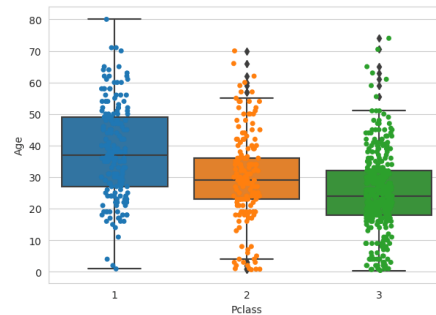
Heatmaps



Bubble plots



Histograms



Box and Whisker plots

And many, many others!



- Pandas is a library for **data manipulation**, particularly when you need to manipulate a spreadsheet-like object (think a super up Microsoft Excel)
- Very useful for cleaning and analyzing data, is the primary tool for data analytics

Key Pandas Features



- Data Importing
 - Can load data from .csv, Excel files, .json, sql, etc.
- Data Cleaning
 - Can quickly find and remove invalid/duplicate data
- Filtering and Slicing
 - Can extract portions of data using conditional logic
 - Like by timestamp, value, etc.
- Merging Datasets
 - Can merge datasets across multiple file formats



- Scikit-learn is the gold-standard machine-learning library for Python
- Note that this is NOT for modern deep learning, more for classical ML like regressions, random forests, clustering

Key Scikit-learn features



- Preprocessing
 - Has many useful functions for normalizing, standardizing, and generally scaling data
- Building models for many forms of ML
 - Classification
 - Identifying which category an object belongs to
 - Ex. Random Forest
 - Regression
 - Predicting a continuous numerical value for an object
 - Ex. Linear regression
 - Clustering
 - Grouping similar objects into larger clusters
 - Ex. K-means clustering

Code Along: Pandas, Matplotlib, and Scikit



1

Make your own CSV

In `make_data.py`, build a messy fake arm log and save it as a CSV. Nothing to download.

2

Load and clean it

In `week3_data.py`, load your CSV with Pandas, clean it, then plot it and fit a line.

3

Find the bad arm

Your plot should show one station's wrist clearly off the line.

YOUR FILES

`make_data.py`

`week3_data.py`

RUN IT (IN ORDER)

```
python make_data.py
```

```
python week3_data.py
```

Code Along: Make the Data



```
import numpy as np
import pandas as pd

rng = np.random.default_rng(217)                    # same "random" data for everyone
n = 300
station = rng.integers(1, 4, n)                      # stations 1, 2, 3
joint = rng.choice(["shoulder_lift", "elbow_flex", "wrist_flex"], n)
cmd = rng.uniform(-90, 90, n).round(1)              # commanded angle (deg)

bad = (station == 2) & (joint == "wrist_flex")      # one arm is miscalibrated
meas = np.where(bad, 0.95 * cmd + 6, cmd) + rng.normal(0, 0.6, n)
temp = 30 + 0.08 * np.abs(cmd) + rng.normal(0, 1, n)

df = pd.DataFrame({"station": station, "joint": joint, "commanded_deg": cmd,
                   "measured_deg": meas.round(2), "temp_C": temp.round(1)})
```

File: make_data.py. One arm is secretly off. Your job: measure exactly how bad, using only the data.

Code Along: Mess It Up and Save



```
# Make it messy, like a real logger
df["measured_deg"] = df["measured_deg"].astype(object)
df.loc[::19, "measured_deg"] = "ERR" # sensor glitches
df.loc[::7, "joint"] = df.loc[::7, "joint"].str.upper() + " "
df.loc[::31, "temp_C"] = 999.0 # error code, not a temp
df = pd.concat([df, df.iloc[:10]], ignore_index=True) # duplicate rows

df.to_csv("so_arm_telemetry_raw.csv", index=False)
print("Saved", len(df), "rows") # Saved 310 rows
```

Run `python make_data.py`. You just made your own CSV, as messy as a real robot log.

Code Along: Load and Clean



```
import pandas as pd

df = pd.read_csv("so_arm_telemetry_raw.csv")
df.info()                                # look before you clean!
print(df["joint"].unique())              # messy names

df["joint"] = df["joint"].str.strip().str.lower()
df["measured_deg"] = pd.to_numeric(df["measured_deg"], errors="coerce")

df = df.drop_duplicates()
df = df.dropna(subset=["measured_deg"])
df = df[df["temp_C"] < 200]
print(len(df))                          # 275 clean rows

wrist2 = df[(df["joint"] == "wrist_flex") & (df["station"] == 2)]
```

New file: week3_data.py. Look before you clean: spot the problems in `df.info()` and `.unique()` first.

Code Along: Plot and Fit



```
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

plt.scatter(wrist2["commanded_deg"], wrist2["measured_deg"], label="Station 2")
plt.plot([-90, 90], [-90, 90], "k--", label="perfect (y = x)")
plt.xlabel("Commanded (deg)"); plt.ylabel("Measured (deg)")
plt.legend(); plt.show()

X = wrist2[["commanded_deg"]] # double brackets: 2D table
y = wrist2["measured_deg"]
model = LinearRegression().fit(X, y)

gain, offset = model.coef_[0], model.intercept_
print(f"measured = {gain:.3f} x commanded + {offset:.2f}") # ~0.95, ~6.0
print((45 - offset) / gain) # command this to land at 45 deg
```

Same file, keep going. **Which station's wrist is off, and by how much?**

Useful properties of matrices



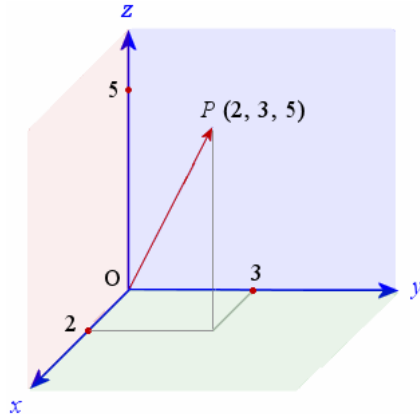
- A matrix is basically just a rectangular 2D array
- Matrices have many useful properties and operations.
- One in particular is **matrix multiplication**
 - We won't go into detail on how to compute a matrix multiplication as you'll usually use **NumPy** to do it
 - (and that's what `ur.linalg` class is for :P)

$$M = \begin{pmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1m} \\ a_{21} & a_{22} & a_{23} & \dots & a_{2m} \\ a_{31} & a_{32} & a_{33} & \dots & a_{3m} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \dots & a_{nm} \end{pmatrix}$$

Rotation Matrices



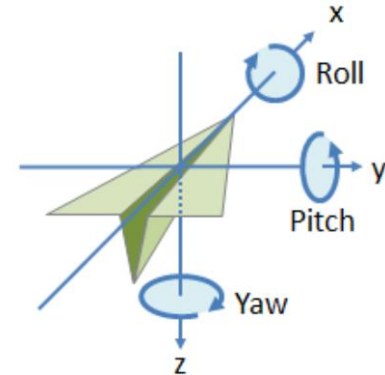
- Rotation Matrices allow you to rotate coordinates by just multiplying the coordinate vector by a matrix
- For example, to rotate the roll, pitch, and yaw of a point/object you can just multiply its coordinate vector by a matrix



$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{bmatrix}$$

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix}$$

$$R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



Picture of Euler Angles

Code Along: Rotations



1

Make your file

Create a new file in your project folder.

2

Type along

Follow the next three slides: 2D rotation, 3D rotation order, then the gripper.

3

Keep this file

You'll use its `fk()` in the hands-on section to check the real arm.

YOUR FILE

`week3_rotations.py`

RUN IT

```
python week3_rotations.py
```

Code Along: 2D Rotation



```
import numpy as np

def rot2d(deg):
    t = np.deg2rad(deg)                # sin/cos want radians!
    return np.array([[np.cos(t), -np.sin(t)],
                     [np.sin(t),  np.cos(t)]])

print(np.round(rot2d(90) @ [1, 0], 3))    # [0. 1.]

R = rot2d(37)
print(np.allclose(R.T @ R, np.eye(2)))   # True: inverse = transpose
print(np.linalg.det(R))                   # 1.0
```

Inverse = transpose: undoing a rotation costs nothing.

Code Along: 3D Rotation Order



```
import numpy as np

def Rx(deg):
    c, s = np.cos(np.deg2rad(deg)), np.sin(np.deg2rad(deg))
    return np.array([[1, 0, 0], [0, c, -s], [0, s, c]])

def Rz(deg):
    c, s = np.cos(np.deg2rad(deg)), np.sin(np.deg2rad(deg))
    return np.array([[c, -s, 0], [s, c, 0], [0, 0, 1]])

p = np.array([1, 0, 0])
print(np.round(Rz(90) @ Rx(90) @ p, 3))      # [0. 1. 0.]
print(np.round(Rx(90) @ Rz(90) @ p, 3))      # [0. 0. 1.] order matters!
```

Try it with a pen: roll then yaw vs. yaw then roll lands in two different places.

Code Along: Where Is the Gripper?



```
from scipy.spatial.transform import Rotation

q = Rotation.from_euler("z", 90, degrees=True).as_quat()
print(q)                                # [0, 0, 0.707, 0.707] as x, y, z, w

L1, L2 = 0.12, 0.14                      # meters: measure your arm!

def fk(t1, t2):
    elbow = rot2d(t1) @ [L1, 0]
    return elbow + rot2d(t1 + t2) @ [L2, 0]

print(fk(0, 0))                          # [0.26, 0] arm straight out
print(fk(90, -90))                       # [0.14, 0.12]

A = np.array([[2, 1, -1], [-3, -1, 2], [-2, 1, 2]])
b = np.array([8, -11, -3])
print(np.linalg.solve(A, b))             # [ 2.  3. -1.]
```

Uses rot2d from two slides back. **This fk() is what you check against the real arm later today.**



TEXAS A&M UNIVERSITY
ROBOTICS TEAM & LEADERSHIP EXPERIENCE

SO-ARM Hands-On

Does the math match the metal?

Before You Start: Calibrate



Same pose
→ **same numbers**

Calibration tells LeRobot where each joint's middle and limits are, so the leader and follower read the same values in the same pose.

Remember Station 2's wrist from the data code along? That's what a bad calibration looks like.

1

Middle pose

Move every joint to the middle of its range, then press Enter. This becomes 0.

2

Full range

Move each joint all the way to BOTH ends, one at a time, then press Enter.

3

Saved by id

LeRobot saves a JSON file named after your --id. Use that same id every time you run the arm.

4

Test with teleop

Move the leader. If the follower copies it smoothly, you're calibrated.

Calibrate: The Commands



```
# Inside the container (docker start -ai lerobot_container)

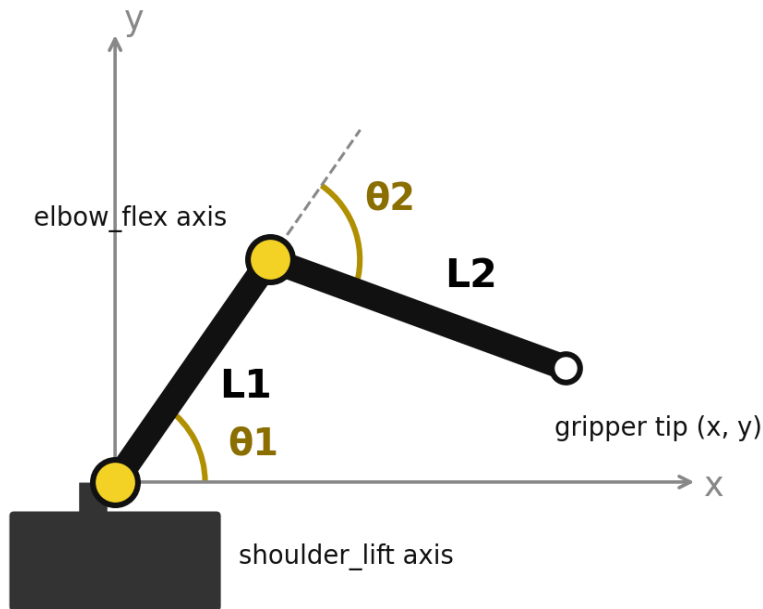
# 1. Follower
lerobot-calibrate \
  --robot.type=so101_follower \
  --robot.port=/dev/ttyACM0 \
  --robot.id=follower_arm

# 2. Leader
lerobot-calibrate \
  --teleop.type=so101_leader \
  --teleop.port=/dev/ttyACM1 \
  --teleop.id=leader_arm

# 3. Test it: the follower should copy the leader
lerobot-teleoperate \
  --robot.type=so101_follower --robot.port=/dev/ttyACM0 --robot.id=follower_arm \
  --teleop.type=so101_leader --teleop.port=/dev/ttyACM1 --teleop.id=leader_arm
```

Use your ports and ids (**follower_arm_2**, **leader_arm_2**, ...). Leave off `--display_data` inside Docker.

Step 1: Measure Your Arm



1

Straighten the arm

Pose the follower so the upper arm, forearm, and wrist all point in one straight line.

2

Measure L1

Calipers from the center of the shoulder_lift servo horn to the center of the elbow_flex horn.

3

Measure L2

From the elbow_flex horn to the gripper tip. Write both lengths in meters.

Step 2: Read Your Joint Angles



```
from lerobot.robots.so_follower import SO101Follower, SO101FollowerConfig

cfg = SO101FollowerConfig(port="/dev/ttyACM0", id="follower_arm", use_degrees=True)
arm = SO101Follower(cfg)
arm.connect()
arm.bus.disable_torque()    # arm goes limp so you can pose it by hand

obs = arm.get_observation()
for joint in ["shoulder_lift", "elbow_flex", "wrist_flex"]:
    print(joint, round(obs[f"{joint}.pos"], 1))

arm.disconnect()
```

Run it inside the container: `docker start -ai lerobot_container`, type `python`, paste.

Use **your** port (check `ls /dev/ttyACM*` from Ubuntu) and the id you calibrated with. Native conda? Use your COM port.

Torque off = no power holding the arm. Support it with your hand first!

Step 3: Find Your Zeros



LeRobot's 0
is not
the math's 0

Calibration sets 0 at the middle of each joint's range. Our FK math wants 0 = pointing straight along x.

1

Find offset 1

Pose the upper arm level (use a phone level app). The shoulder_lift reading is offset1.

2

Find offset 2

Line the forearm up with the upper arm. The elbow_flex reading is offset2.

3

Check the sign

Raise the arm a bit. Reading went down? Your sign is -1.

$$\theta = \text{sign} \times (\text{reading} - \text{offset})$$

Step 4: Predict, Then Measure



1

Pose

Set the arm in any pose. Keep wrist_flex straight.



2

Read

Run the Step 2 code.
Write down the angles.



3

Predict

Convert readings with Step 3, then run your fk().



4

Measure

Ruler from the shoulder_lift axis: x across, y up.

```
error = np.linalg.norm(predicted - measured)
```

Do three different poses. Big error? Recheck your zeros and signs before blaming the math. Consistent error in one direction usually means an offset is wrong.

Common Mistakes



X Degrees into np.sin

np.sin(90) is not 1.
Convert with np.deg2rad first.

X * instead of @

A * B multiplies element by element. Rotations need A @ B.

X Wrong rotation order

Rz @ Rx is not Rx @ Rz.
The rightmost rotation happens first.

X Not assigning back

df.dropna() returns a NEW DataFrame. Write df = df.dropna().

X Single brackets for X

Scikit wants a 2D table: df[["col"]], not df["col"].

X Rushing calibration

Move every joint to BOTH ends, or its range gets cut short.

X Mismatched ids

Calibrate as follower_arm, run as follower_arm too.
Same id every time.

X Trusting LeRobot's zeros

Calibration's 0 is mid-range, not level. Find your offsets.

Deliverable



1

NumPy

week3_numpy.py runs and every output matches the slide comments.

2

Data

Data cleaned to 275 rows, and you found the bad wrist's gain and offset.

3

Rotations

fk(90, -90) gives [0.14, 0.12], and both rotation orders print as shown.

4

Hands-On

Calibrate both arms, get teleop working, then predict vs. measure 3 poses. Send your lead a photo and your errors.

All four done = Week 3 complete. Checkpoint 4 is the one that counts most.